

A Constructive Method to Build Many Valid Initial Solutions for the Traveling Tournament Problem

Guilherme Nakahata^{1,3}[0000–0001–6469–8653], Florian
Richoux¹[0000–0003–4693–379X], Daan van den Berg²[0000–0001–5060–3342], and
Claus Aranha³[0000–0003–1390–7536]

¹ AIST, Japan

{guilherme.nakahata / florian.richoux}@aist.go.jp

² Vrije Universiteit Amsterdam, The Netherlands

d.van.den.berg@vu.nl

³ University of Tsukuba, Japan

caranha@cs.tsukuba.ac.jp

Abstract. The Traveling Tournament Problem is infamous for not having fast and high quality methods for mutation, crossover and creating valid random initial solutions, making it particularly difficult for metaheuristic search. As a step forward, we propose a constructive method capable of creating many valid Traveling Tournament Problem solutions. This method is a modification of the circle method to make a schedule for single round-robin tournaments, originally proposed by Reverend Kirkman in 1847. It runs in linear time in the number of teams. The parametrized version of the method can produce up to $2n \cdot n!$ different valid solutions, and we can increase this number up to $(2n - 1)2n \cdot n!$ valid solutions with a simple permutation of rounds in produced solutions. These results can contribute significantly to the adaptation of population-based metaheuristics to this problem.

Keywords: The Traveling Tournament Problem · Valid solutions · Initial solutions · Evolutionary computing · Constructive method.

1 Introduction

The Traveling Tournament Problem (TTP) is a variant of the Traveling Salesman Problem (TSP) that includes scheduling aspects [9]. Given n teams, one must make a tournament schedule of $2(n - 1)$ rounds, where each team will play a game against another team each round. The objective of the TTP is to find a schedule that minimizes the total distance travelled by all teams, while respecting the following three constraints:

1. Double round-robin (*doubleRoundRobin*): Each team will play every other team exactly twice, once at home and once away,

2. Maximum streak ($maxStreak=k$): No team will play more than k games in a row as Home or as Away. In this paper, we will set $maxStreak = 3$, which is the default value for most TTP studies,
3. No repeats ($noRepeat$): No team will play against the same opponent in two consecutive rounds.



Fig. 1: Two examples of TTP instances. The rows indicate rounds, while the columns indicate the matches of each team. The yellow airplane/house icon indicates an "away" or "home" game, respectively. Left: A valid 4-team TTP tournament. Right: An invalid TTP tournament, having a $noRepeat$ violation in column 2, a $maxStreak=3$ violation in column 4, and $doubleRoundRobin$ violations in rounds (rows) 2, 3 and 4.

These three constraints, $doubleRoundRobin$, $maxStreak = 3$ and $noRepeat$, illustrated in Figure 1, make the problem much harder than TSP, whose only constraint is that the tour must visit each city in the problem exactly once (i.e., form a Hamiltonian cycle). Section 2 details the difficulties caused by these extra constraints, and how the related literature has approached them so far.

In particular, the extra constraints make it much more difficult to generate *valid* TTP solutions, either from scratch, or from variation operators such as crossover or random sampling. This causes the problem to be specially difficult to solve for metaheuristics which rely on a valid initial population.

This is the problem that we address in this paper, by proposing an algorithm to generate valid solutions for a given instance of the TTP. The key idea is to lay the teams on a circle and create matches between teams on opposite sides. Successive rounds are then created by rotating the teams while keeping one *pivot*

team fixed. Multiple valid solutions can be created by changing the pivot and the order of the teams.

We give a detailed description of the algorithm in Section 3, and proof of its correctness is Section 4. We also made a canonical implementation, in Rust, which is publicly available on Zenodo⁴ and GitHub⁵. This library can be used to generate initial populations of valid solutions for metaheuristic methods aiming at finding *optimal* solutions for the TTP.

2 Preliminaries

2.1 Application and History of the TTP

The objective of the TTP is the same as that of the TSP: To minimize travel time. In the United States, where travel distances are quite significant and Major League Baseball is a tournament that hosts vast sportive and financial interests, and even slight improvements to a schedule can make significant differences [30,23]. For many years, the scheduling for this tournament was done on paper, by Henry and Holly Stephenson, a married couple from Massachusetts, but was eventually rigorously formulated in 2001 by Kelly Easton, George Nemhauser and Michael Trick in their seminal publication on the problem [10].

But a lot has changed since then. In the last decade, we started recognizing that although the Traveling Tournament Problem is very similar to the Traveling Salesman Problem, it is in some ways also very different. Much harder, as it appears, even though both problems are classified as NP-hard [3,4,25]. Given this qualification, both problems require exponential time to solve for an exact solution. It is true that the exponential complexity function of the TTP might be a little steeper due to the double-quadratic nature of its tournament table, but this is not the biggest difference that sets the TTP apart as being harder than the TSP.

2.2 On the Difficulty of the Constraints

The three constraints of the TTP (*doubleRoundRobin*, *maxStreak*, and *noRepeat*) make it a significantly more difficult problem than the TSP.

The *doubleRoundRobin* constraint [10] requires a schedule for an even number n of sports teams in which every team plays every other team twice: Once at home, and once away. It is seldom noted, but important, that teams should play one another symmetrically, meaning that if Team A plays at home against Team B, Team B should also play at away against Team A. This may seem like a trivial requirement, but past implementations do not always account for this when applying crossover or mutation operators to schedules, possibly corrupting experimental outcomes.

⁴ <https://zenodo.org/records/17527890>

⁵ https://github.com/GuilhermeNkht/TTP_ConstructiveMethod/tree/1.001

The *maxStreak* constraint requires that teams do not play more than k consecutive games at home (or away). In most implementations, *maxStreak* = 3, meaning no more than 3 consecutive home or away games can be scheduled for any team. It is thought, though not proven to the best of our knowledge, that the magnitude of the *maxStreak* parameter critically influences the hardness of the problem [28].

Finally, the *noRepeat* constraint enforces that if Teams A and B play against each other in round r , they cannot play against each other in round $r + 1$.

2.3 Metaheuristic search in the TTP

Although the TTP and the TSP are both NP-hard, the TTP's constraints has so far practically disqualified it for (meta)heuristic population based optimizers. A plethora of metaheuristic search algorithms such as simulated annealing, local search, genetic algorithms and others, have been applied for the TSP with more or less success [1]. The TTP, on the other hand, has very few, and they all suffer from the same problems:

1. There is no known deterministic polynomial time mutation for the TTP that connects the entire search space. Most mutation types either very often produce invalid individuals or do not connect the entire search space. Changing the home/away designation or swapping columns are examples of the latter. Purposefully designing a mutation that *does* connect the entire search space appears not to have been attempted, not even with invalid-resampling techniques – which in itself might take stochastic exponential time.
The contrast with the related TSP could not be starker: Every non-selective operation that mutates the permutation of cities into another permutation of cities unbiasedly connects the entire search space. Examples are moving a random segment of 5 cities, or swapping 2 or more cities at random. Moreover, all these mutations can be executed in deterministic constant time. These mutative luxuries are unavailable to the TTP.
2. There is no known deterministic polynomial time crossover operation for the TTP. This is hardly surprising given the absence of a similar mutation operator; even for the much easier TSP, many crossover operators require some kind of repair mechanism [13]. Only for the easiest hard problems, like the partition problem, crossovers can be performed without repair methods in deterministic polynomial time.
3. There is no known deterministic polynomial time algorithm for sampling valid random solutions for TTP. This is perhaps the most surprising complication the TTP presents: We cannot randomly generate an initial solution for the TTP to use as a starting point for simulated annealing or local beam search.

The best we currently have for the TTP is stochastic exponential time, meaning we *can* uniform randomly sample an initial solution, but its expected number of constraint violations increases so fast, that even randomly

resampling cannot be done repeatedly, as it is expected to take an exponential number of samples in n before obtaining just one single valid solution [29,27,28,12].

Again, compare this to the TSP, where a fully uniform random sample from the search space can be created from repeatedly picking available cities at random from a list, and then removing that city: It gives us a uniformly chosen initial TSP solution in deterministic linear time.

Interestingly enough, the TTP appears not to be alone in suffering these problems, as HP-protein folding also has no violation-free mutations, crossovers or initialization procedures [15,14]. But despite these complications, metaheuristic attempts have nonetheless been made on the TTP. Biajoli and Lorena applied a combination of a genetic algorithm together with simulated annealing to a mirrored-TTP (mTTP) in which the *doubleRoundRobin* constraint is satisfied by repeating the first half of the schedule with home/away designations reversed [5], and their population is initialized using the circle method [7,8]. Their mutations (swapping the home/away roles for a match, swapping two matches, or swapping the entire match plan for the two teams) can lead to invalid solutions though, and are unlikely to connect the entire search space.

In another simulated annealing study from 2006, Anagnostopoulos et al. treat only the *doubleRoundRobin* as a hard constraint, while *noRepeat* and *maxStreak* are treated as soft constraints [2]. Soft constraints may be violated as the algorithm traverses the search space, mutating by swapping rounds (either partially or whole), home/away designations, or opponents. The big open questions however are whether these mutations connect the entire search space, and moreover, how long the algorithm remains in invalid space – it might be forever. Interestingly, these authors use a recursive function to “generate random [initial] schedules satisfying the hard constraints”. We assume this works quite well, but it also takes deterministic exponential time, and might not be random.

Numerous studies investigated TTP with diverse metaheuristics algorithm, such as PSO [24], genetic algorithm [21,16], Artificial Bee Colony algorithm [22], Ant Colony Optimization [26], etc. However, these works illustrate the community’s inability to find good mutations, crossovers and initialization procedures. These operators might very well not exist for the TTP, and one reason might be that valid TTP-solutions are almost maximally different from one another [19,18]. But much like the $P \stackrel{?}{=} NP$ problem, the non-existence of efficient procedures should not stop us from looking for them.

The metaheuristic community is not the only one working on generating solutions. The community working on approximation algorithms for TTP has proposed many constructive methods, often based on two methods from late 2000s / early 2010s, respectively the modified circle method from Fujiwara et al [11] and the Hamiltonian cycle construction from Yamaguchi et al [31]. Since the method we propose is also about generating valid solutions, we describe here briefly how these algorithms work.

Both methods are based on the circle method. The modified circle method is calling the circle method to build a single round-robin schedule with n teams,

where home and away venues are assigned following a fixed home/away pattern, with a few exceptions that we will not detail here. A double round-robin schedule is then obtained by interweavingly concatenating games from the single round-robin schedule $S = (r_1, r_2, \dots, r_{n-1})$, *i.e.*, a series of $n - 1$ rounds built as above, with the same schedule $\bar{S}$ with reverse venues, following the scheme $(r_1, r_2, r_3, \bar{r}_1, \bar{r}_2, \bar{r}_3, \bar{r}_4, \bar{r}_5, \bar{r}_6, r_4, r_5, r_6, \dots)$.

The Hamiltonian cycle construction is mixing the circle method with an almost shortest Hamiltonian cycle. The circle method is used twice to fix slots of a double round-robin schedule, first without considering home and away assignments. Then, such an assignment is made constructively by repeating k home games followed by k away games, for a given k , with an exception of the two last slots that are not repeating this pattern. Finally, the assignment of a team to each slot is done by using a short Hamiltonian cycle, computed with Christofides approximation algorithm [6] for the Traveling Salesman Problem.

However, these works mainly focus on lowering the approximation bound, and do not really deal with offering simple constructive methods. Moreover, such methods may generate numerous valid solutions usable for metaheuristics, but their authors did not investigate how many solutions are generalizable.

In the current study, we will present a simpler method to create many valid initial solutions to the TTP. Even though it is not the polynomial time fully uniform sampling method we desire, it is a firm step forward in the right direction.

3 Constructive method

Our constructive method is a modification of the circle method, a.k.a. the polygon method, to make a schedule for single round-robin tournament, proposed by Reverend Kirkman in 1847 [17]. This original circle method assigns who plays whom and when, but does not decide anything about allocating venues. The circle method in its modern form has been proposed by de Werra [7,20], to also consider venue allocation.

Our method is given in Algorithm 1, and illustrated with Figure 2. Its principle is simple when explained in a graphical manner (for $n = 6$ is our illustration):

1. Write n teams, with n even and greater than 2, in a circle.
2. Fix the team on the top (Fig. 2a.)
3. Make matches (home, away) by drawing diagonal upward arrows away $\rightarrow$ home, from left to right. Fig. 2b depicts matches (1,6), (2,5) and (3,4) for round 1.
4. Rotate teams to the right, except the fixed team (Fig. 2c.)
5. Make matches by drawing diagonal downward arrows from right to left, like in Fig. 2d.
6. Rotate again to the right.
7. Repeat $n - 1$ times from Step 3.

The reader can observe that Algorithm 1 actually outputs mirrored solutions, since games in the last $n - 1$ rounds are simply the reversed games from the first $n - 1$ rounds.

Algorithm 1: BUILDTTPSOLUTION

Input: A list $x = \Pi(1, 2, \dots, n)$, being a permutation of n teams, with $n > 2$ an even number.

Output: A valid schedule for TTP.

```

1 Matches  $M \leftarrow \emptyset$ 
2 for (  $r = 1$ ;  $r \leq 2(n-1)$ ;  $r = r + 1$  ) {
3   if  $r$  is odd then  $M \leftarrow M \cup \{(x[n-1], x[n-2]), (x[0], x[n-3]), (x[1], x[n-4]), \dots, (x[\frac{n}{2}-2], x[\frac{n}{2}-1])\}$ 
4   else  $M \leftarrow M \cup \{(x[n-2], x[n-1]), (x[n-3], x[0]), (x[n-4], x[1]) \dots, (x[\frac{n}{2}-1], x[\frac{n}{2}-2])\}$ 
5   Rotate( $x[0..n-2]$ ) //  $tmp \leftarrow x[n-2]$ ,  $x[n-2] \leftarrow x[n-3]$ ,  $x[n-3] \leftarrow x[n-4]$ ,  $\dots$ ,  $x[0] \leftarrow tmp$ .  $x[n-1]$  is fixed.
6 return  $M$ 
```

4 Soundness

First, we observe that, since Algorithm 1 only contains a for-loop with $2(n-1)$ iterations, it necessarily terminates. To show that solutions by Algorithm 1 are valid, we need to show that they satisfy the three constraints *doubleRoundRobin*, *maxStreak=3* and *noRepeat*. Note that satisfying *maxStreak=2* implies satisfying *maxStreak=3*.

Proposition 1. *Let $n \geq 4$, with n an even number of teams. Algorithm 1, applied on a list of n teams, outputs a schedule satisfying the *maxStreak=2* constraint (*maxStreak=3* if $n = 4$).*

Proof. Consider $n \geq 6$ even. We can split the list of teams into two blocks: From index 0 to $\frac{n}{2} - 2$, and from index $\frac{n}{2} - 1$ to $n - 2$. We include the fixed team at index $n - 1$ into the first block. This way, each round, teams in one block can only have a game against a team from the other block. Visually, this gives two fixed half circles like illustrated in Figure 3, in green and pink, separated by a “pivot line”.

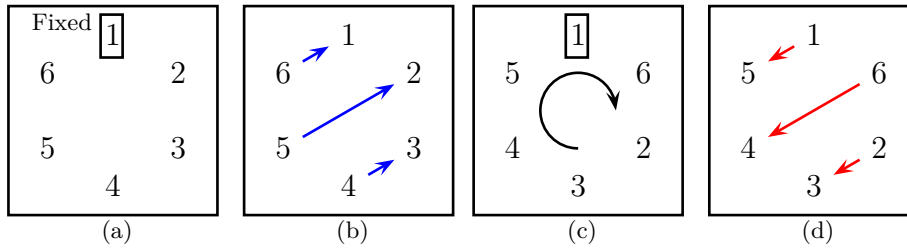


Fig. 2: Illustration of the main steps: Place teams on a circle and fix one; match-making (arrows); rotate non-fixed teams; matchmaking (reversed arrows).

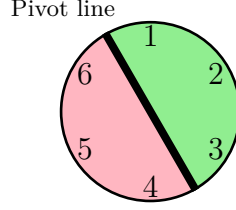


Fig. 3: Visual for the proof of Proposition 1.

A team rotating within the same block will alternate each round between home and away matches, by construction (Lines 3 and 4 of Algorithm 1). The only moments where a team will play twice in a row at home or away is when this team rotates from one block to the other. Visually, this happens when a team cross the pivot line after a rotation. For instance, assume that Team t at position $n - 2$ is playing home (or away, respectively) against the team at position $n - 1$ in round r (Line 4, or respectively Line 3). Thus, at round $r + 1$, Team t will be at position 0 and will play home again (or away again, respectively) against the team at position $n - 3$ (Line 3, or respectively Line 4). Same thing if Team t is a position $\frac{n}{2} - 2$ in round r . Since no teams have games more than two times in a row at home or away, the $\text{maxStreak}=2$ constraint is satisfied when $n \geq 6$ holds.

Consider now $n = 4$. By construction, the team at the fixed position 3 will always alternate between home and away games (Lines 3 and 4). The team at position 1 will rotate into the same block and will alternate between home and away. The team at position 0 will cross the pivot line once before doing one rotation within the same block, and thus will play home or away two games in a row. Only the team at position 2 will play home or away three times in a row since it will cross two times in a row the pivot line. The $\text{maxStreak}=3$ constraint is satisfied. $\square$

Proposition 2. *Let $n \geq 4$, with n an even number of teams. Algorithm 1, applied on a list of n teams, outputs a schedule satisfying the NoRepeat constraint.*

Proof. Assume, for sake of contradiction, that we have matches (t_1, t_2) in round r and (t_2, t_1) in round $r + 1$. Let $x[i] = t_1$ and $x[j] = t_2$ hold in round r , and let denote the match (t_1, t_2) by $(i, j)_r$. Without loss of generality, suppose r is odd, and then $r + 1$ is even.

We consider the same blocks of List x as in the proof of Proposition 1. The first block goes from index 0 to $\frac{n}{2} - 2$, with $n - 1$ in addition, and the second from index $\frac{n}{2} - 1$ to $n - 2$.

In round r , since Teams t_1 and t_2 have a game together, they necessarily belong to a different block. Since r is odd and we have the match (t_1, t_2) , it means that t_1 belongs to the first block (the “home” block in odd rounds), and t_2 to the second (the “away” block in odd rounds). Then, $0 \leq i \leq \frac{n}{2} - 2$ and $\frac{n}{2} - 1 \leq j < n - 2$ hold. We will see later why we cannot have neither $i = n - 1$ nor $j = n - 2$.

We have the match (t_2, t_1) in round $r+1$. Necessarily, we have now $x[i+1] = t_1$ and $x[j+1] = t_2$ in round $r+1$. However, since $r+1$ is even, with Line 4, we have the matches $(j+1, i-1)_{r+1}$ and $(j-1, i+1)_{r+1}$. Having the match (t_2, t_1) in round $r+1$ implies that $i-1 = i+1$ and $j-1 = j+1$ hold, which is only possible when $n = 2$. Contradiction.

Now, let's consider the two cases we excluded above. Assume that Team t_1 is at position $n-1$ in round r . Since this position is fixed, t_1 will stay at position $n-1$ in round $r+1$. Playing twice in a row against Team t_2 implies that t_2 also remains at the same position in round $r+1$, which is only possible when $n = 2$. Contradiction.

Finally, assume that Team t_2 is at positions $n-2$ in round r . Thus, Team t_1 is at position 0. In round $r+1$, Team t_2 rotates to finish at position 0, and t_1 must be placed at position $n-2$ to have the game with t_2 . However, this is only possible when $n = 3$, which raise a contradiction. $\square$

Lemma 1. *Let $n \geq 4$, with n an even number of teams. Algorithm 1 applied on n outputs a schedule where, for any sub-schedule of $n-1$ rounds in a row, each team plays against each other team exactly once.*

Proof. First, let observe that this is obviously the case for the team $x[n-1]$, since it is fixed and all other teams are rotating one step further in the list. Moreover, since we have an odd number of rotating teams, and since we always alternate between home and away matches for $x[n-1]$ (Lines 3 and 4), the team at $x[n-1]$ will play all possible games $(t, x[n-1])$, $(x[n-1], t)$, for each other team t , during the full run of the algorithm. Reciprocally, it implies that all other team t will exactly play twice against the team $x[n-1]$, one time at home and one time away, during $2(n-1)$ rounds. Second, let observe that each team that is not at index $n-1$ will necessarily be placed exactly twice at each index $i \in \{0, \dots, n-2\}$ during the run of the algorithm.

Without loss of generality, assume that x is initialized such that $x[i] = i$ holds, for all $i \in \{0, n-1\}$. The input of Algorithm 1 indicates that team numbers are going from 1 to n , but it makes the proof easier to consider here team numbers from 0 to $n-1$, without changing anything to the algorithm. Now, after some iterations of the loop in Line 2, let Team i be at position 0, i.e., $x[0] = i$ holds. Then, Team i will play against either Team $i-2$ if $i \geq 2$, or Team $i+n-3$ otherwise. One iteration later, Team i is at position 1 and will play against Team $i-4$ if $i \geq 4$, or Team $i+n-5$ otherwise. Some iterations later (less than $n-2$ iterations), Team i is at position j and will play against Team $i-2j$ if $i \geq 2j$, or Team $i+n-2j-1$ otherwise.

Now, suppose that there is a Team k that never plays against Team i during $n-1$ rounds in a row. Thus, necessarily, there exists a Team l such that Teams i and l will play at least twice during these $n-1$ rounds. Assume that, in two different rounds, two games between Teams i and l occur when Team i is at position j_1 and j_2 , with $j_1 < j_2$. If $i \geq 2j_2$, it implies $i \geq 2j_1$. Thus, we have $l = i - 2j_1$ and $l = i - 2j_2$, which is a contradiction with $j_1 < j_2$. Thus, we must have $i < 2j_2$. If $i < 2j_1$ holds, we have the same impossible situation where

$l = i + n - 2j_1 - 1$ and $l = i + n - 2j_2 - 1$. Thus, we must have $i \geq 2j_1$. In that case, we have $l = i + n - 2j_2 - 1$ and $l = i - 2j_1$. Thus, we have $i - 2j_1 = i + n - 2j_2 - 1$, then $2j_1 = 2j_2 + 1 - n$, then $j_1 = j_2 + \frac{1}{2} - \frac{n}{2}$, which is impossible because it implies that j_1 is not a natural number. Contradiction.

Since, for all i, l and k , we cannot have two teams i and l playing two games each other in $n - 1$ consecutive rounds, and we cannot have a team k that is not playing against a team i in $n - 1$ consecutive rounds, we have necessarily all teams playing against each other teams in $n - 1$ consecutive rounds. $\square$

Proposition 3. *Let $n \geq 4$, with n an even number of teams. Algorithm 1, applied on a list of n teams, outputs a double round-robin schedule.*

Proof. By Lemma 1, we know that we have a single round-robin schedule for the $n - 1$ rounds in a row. Thus, during the full season of $2(n - 1)$ rounds, each team will play twice against each other teams.

Since all teams return to their initial position after $n - 1$ rotations, with $n - 1$ being odd, and since Algorithm 1 is alternating at each iteration the direction home/away of matches, we have necessarily all teams playing two games against each other team, alternating a game at home and away. We have a double round-robin schedule. $\square$

Proposition 4. *Let $n \geq 4$, with n an even number of teams. Algorithm 1, applied on a list of n teams, outputs a valid schedule for the Traveling Tournament Problem.*

Proof. Consequence of Propositions 1, 2 and 3. $\square$

5 Number of generalizable schedules

Since the output of Algorithm 1 only depends of the input, *i.e.*, how List x has been initialized, the algorithm can produce $n!$ different schedules.

Notice that other schedules may be generated if we slightly modifies the algorithm by fixing a different position in List x . If we add a parameter $0 \leq p \leq n - 1$ to the algorithm, such that we fix the team $x[p]$, instead of constantly fixing $x[n - 1]$, this does not have any impact on properties of the produced schedules, and allow us to generate up to $n \cdot n!$ different schedules, regarding p and the random initialization of x .

We can also add a second parameter, indicating whether we start by “upward” matchmaking or “downward” matchmaking, to take intuition of Figure 2. In practice, it can be done by replacing ‘*r is odd*’ in Line 3 with ‘*r is even*’. This doubles the number of valid solutions we can produce, *i.e.*, $2n \cdot n!$.

Finally, we can make a simple perturbation on a generated solution to get even more valid solutions, for $n \geq 6$. Let denote respectively by f_r and l_r the first and last match of a round r . For each Solution s , we can generate $2(n - 1)$ additional valid solutions by swapping two rounds r_1 and r_2 . Assume that we have $f_{r_1} = (t, 1)$ (respectively $f_{r_1} = (1, t)$), with the fixed team 1 and for some

team $t \in \{2, \dots, n\}$, and that $l_{r_1} = (u, z)$ holds (respectively $l_{r_1} = (z, u)$). Thus, we can swap r_1 with the round r_2 such that $f_{r_2} = (1, u)$ holds (respectively $f_{r_2} = (u, 1)$). The obtained solution s' remains a valid solution, satisfying the three constraints *doubleRoundRobin*, *maxStreak=3* and *noRepeat*.

Proposition 5. *Consider $n \geq 6$ holds, and let s' be a solution obtained from s by swapping rounds r_1 and r_2 as explained above. Solution s' is a valid solution of TTP.*

Proof. It is easy to see that the *doubleRoundRobin* constraint is preserved by round permutations. We will focus on the preservation of *noRepeat* and *maxStreak*.

Observe that there are $\frac{n}{2} - 1$ rounds separating r_1 to r_2 . Indeed, by construction, the fixed Team 1 is placed at position $n - 1$, and Team u is placed at the opposite of Team 1 on the circle, at position $\frac{n}{2} - 1$, if we consider that the first position is position 0. Thus, Team u is placed for the last match in Round r_1 , and needs to climb up to the first match in Round r_2 to play against Team 1. This takes $\frac{n}{2} - 1$ iterations of Algorithm 1: At each iteration, Team u is climbing up one by one the match order, due to the circular rotation of teams, to finally reach position $n - 2$.

Necessarily, all teams other than 1 and u will cross the pivot line (Fig. 3) during these $\frac{n}{2} - 1$ iterations. The fact that $\frac{n}{2} - 1$ is odd, and that all teams but Teams 1 and u cross the pivot line once in $\frac{n}{2} - 1$ iterations, imply that all teams other than 1 and u will play a game on round r_2 with the same place designation than on round r_1 , i.e., away on r_2 if it was away on r_1 , home on r_2 if it was home on r_1 . Only Teams 1 and u , who didn't cross the pivot line, will have their place designation changed, due to the odd number of iterations.

This prevents the swap of r_1 and r_2 of violating the *noRepeat* constraint, since all teams but Teams 1 and u keep the same place designation. The only problem could come from games between Team 1 and u , but since it takes exactly $n - 1$ rounds for Team 1 to play against the same opponent, swapping two rounds separated from $\frac{n}{2} - 1$ rounds cannot violate the *noRepeat* constraint.

Finally, let's show that swapping r_1 and r_2 will neither violate *maxStreak=3*. We just saw that, after $\frac{n}{2} - 1$ iterations, all teams but Teams 1 and u keep the same place designation. Thus, swapping r_1 and r_2 will not change the maximal streak of these teams. The only teams which could violate this constraint are Team 1 and u . However, it is easy to see that it cannot be the case.

For Team u , observe that, since we have $n \geq 6$, we know from Proposition 1 that s satisfies the constraint *maxStreak=2*: A team is assigned twice to the same place designation only when it crosses the pivot line. After swapping r_1 and r_2 , we obtain Solution s' with rounds $r'_1 = r_2$ and $r'_2 = r_1$. On r_1 in s , Team u just crossed the pivot line and was assigned to the same place designation than on the round just before r_1 (or the last round if r_1 is the very first round). Thus on r'_2 in s' , Team u is assigned to the same place designation than on the round just after r'_2 (or the first round if r'_2 is the very last round). We still have *maxStreak=2* satisfied. We can apply the same reasoning for r'_1 in s' .

For Team 1, it is changing its place designation at each round. Thus, swapping r_1 and r_2 spaced by $\frac{n}{2} - 1$ rounds will induce rounds r'_1 and r'_2 in s' such that

Team 1 share the same place designation with games on rounds before and after r'_1 and r'_2 . We have then $maxStreak=3$. $\square$

Corollary 1. *This swapping method creates up to $2(n-1)$ new solutions from a solution s , i.e., one for each round. Thus, we can create up to $2(n-1)2n \cdot n!$ additional valid solutions. Together with the $2n \cdot n!$ solutions the parametric version of Algorithm 1 can generate, it gives an upper bound of $2n \cdot n! + 2(n-1)2n \cdot n! = (2n-1)2n \cdot n!$ valid solutions we can easily generate, with an even number $n \geq 6$.*

6 Empirical Analysis of Generated Schedules

To evaluate the performance of the proposed method, we conducted experiments on the NL8, NL10, NL12, NL14, and NL16 instances, where $n \in \{8, 10, 12, 14, 16\}$, from the dataset introduced by Easton et al. [10]. These instances correspond to National League distance matrices, where distances are measured by air distance between cities. This dataset is widely used in the literature, and our goal is to analyze the diversity of the schedules produced by the constructive procedure.

For each instance, the algorithm was executed 10,000 times, with Team 5 fixed and random values generated using seed 2025. The experiments were run on an Intel Core i5-8250U CPU @ 1.60 GHz with 16 GB RAM, where the constructive phase required between 1 and 10 minutes to generate 10,000 solutions, depending on the instance size. To give a point of comparison, the recent *random backtracking* method from Løyen et al. [18] generates 10,000 solutions in several hours for instance sizes $n \in \{4, 6, 8, 10\}$, up to 40 hours for $n = 10$.

Figure 4 presents the distributions of total travel distances obtained from the generated schedules. The differences in shape and dispersion indicate that the method explores the solution space, instead of concentrating around a limited subset of the search space. Moreover, the distributions are well approximated by normal curves, making it possible to estimate their parameters via regression for larger instances.

Table 1 summarizes the performance of the constructive method against the best bounds in the literature. As expected, the minimum and median travel distances remain above the lower bounds, which is reflected in the reported gaps. Still, the method consistently produces a wide range of valid schedules within each instance, instead of converging to a single solution point. This variability is important, because the constructive phase can start with a high quality starting points that substantially improve the performance of subsequent metaheuristic search.

To examine how the distribution characteristics evolve with league size, we fitted quadratic regression models to the empirical mean μ and standard deviation σ obtained for $n \in \{8, 10, 12, 14, 16\}$. The resulting curves, shown in Figure 5, indicates that the mean and standard deviation increases approximately quadratically as the number of teams grows, which aligns with the combinatorial expansion of the possible matching games space. This suggests that, while the absolute

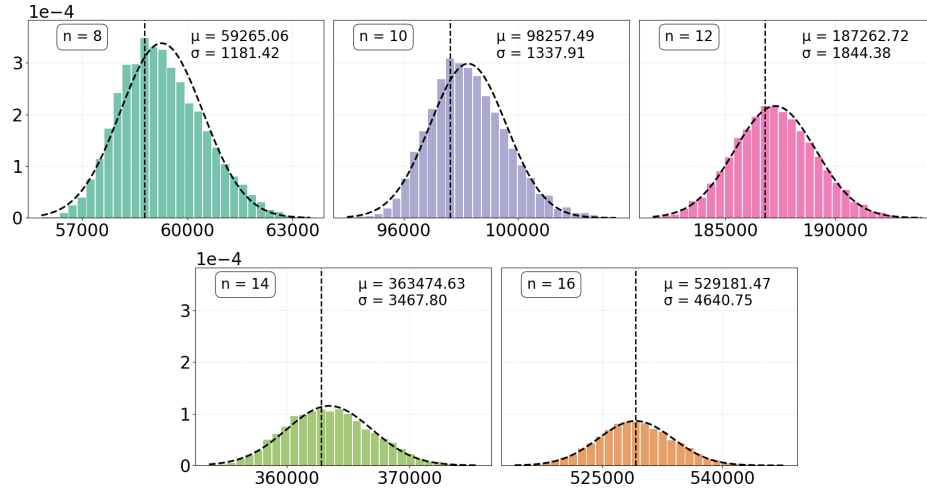


Fig. 4: Distance distributions for instances with $n \in \{8, 10, 12, 14, 16\}$ teams. The dashed vertical line indicates the mean of each distribution. The displayed values of μ and σ correspond to the mean and standard deviation of the sampled distances, and the overlaid curve represents the normal density function using these parameters.

scale of feasible solutions becomes larger, the relative dispersion remains stable, meaning that the solution space does not collapse into a narrow region.

The results in Table 1 show that the travel distances obtained by the constructive method are higher than the best known values in the literature. This is expected: Since the method does not perform iterative improvement, it does not attempt to refine a tournament schedule once it is generated. Its purpose is simply to produce feasible solutions efficiently, without the need for additional constraint handling mechanisms.

Even so, the sets of solutions obtained show variation in their solution distances and meaningful structural differences in their match allocations, as the constructive procedure does not generate the same schedule twice. This is an important property for metaheuristic optimization, where the quality of the final result may depend on the diversity of the initial population. Constructive procedures that produce nearly identical schedules tend to drive metaheuristics toward premature convergence.

In contrast, the proposed method generates large collections of feasible schedules without requiring additional constraint handling strategies or parameter tuning. The variation observed in both the mean and the standard deviation of travel distances indicates that generated solutions are scattered throughout the search space, rather than collapsing to a narrow region.

Table 1: Comparison of our results with the state of the art bounds, including minimum and median values, best lower and upper bounds, and the percentage gap to the best known upper bound.

Instance	Minimum	Median	Best LB	Best UB	Gap (%)
NL8	55,837	59,166	39,721	39,721	40.57
NL10	93,992	98,155	59,436	59,436	58.14
NL12	181,697	187,217	108,629	115,072	57.90
NL14	353,679	363,339	183,354	203,407	73.88
NL16	514,114	529,048	249,500	271,476	89.38

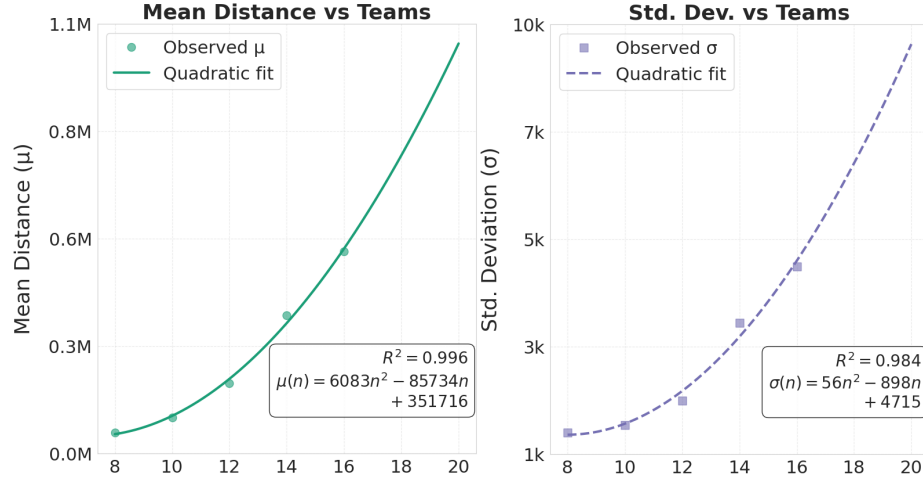


Fig. 5: Quadratic regression of the empirical mean (μ) and standard deviation (σ) of the distance distributions as functions of n . The curves show the fitted models with their R^2 values.

7 Conclusion

It is well known that generating an initial population of valid solutions for the Traveling Tournament Problem is very difficult, due to its severe constraints. In this paper, we proposed a constructive method to generate many valid Traveling Tournament Problem (TTP) solutions, and provide the proofs of its soundness. Our method can generate $2n \cdot n!$ different valid solutions, which can be extended up to $(2n-1)2n \cdot n!$ by a simple perturbation of the produced solutions. It runs in linear time $O(n)$, with n the number of teams, and have been implemented and tested on well-known instances in the literature. It takes from 1 to 10 minutes to generate 10,000 solutions on these instances, where n varies from 8 to 16. We believe our result could help the usage of population-based metaheuristics by providing a valid initial population.

References

1. Alkhalifa, R., Alkhomayes, F., Almazroua, B., Alhaidan, D., Alothman, M., Al-muhaidib, J.: A comparative review of parallel exact, heuristic, metaheuristic, and hybrid optimization techniques for the traveling salesman problem. arXiv preprint arXiv:2505.18278 (2025)
2. Anagnostopoulos, A., Michel, L., Hentenryck, P.V., Vergados, Y.: A simulated annealing approach to the traveling tournament problem. *Journal of Scheduling* **9**(2), 177193 (Apr 2006). <https://doi.org/10.1007/s10951-006-7187-8>, <http://dx.doi.org/10.1007/s10951-006-7187-8>
3. Bhattacharyya, R.: A note on complexity of traveling tournament problem. *Optimization Online* **2480** (2009)
4. Bhattacharyya, R.: Complexity of the unconstrained traveling tournament problem. *Operations Research Letters* **44**(5), 649–654 (2016)
5. Biajoli, F.L., Lorena, L.A.N.: Mirrored traveling tournament problem: An evolutionary approach. In: Sichman, J.S., Coelho, H., Rezende, S.O. (eds.) *Advances in Artificial Intelligence - IBERAMIA-SBIA 2006*. pp. 208–217. Springer Berlin Heidelberg, Berlin, Heidelberg (2006)
6. Christofides, N.: Worst-case analysis of a new heuristic for the travelling salesman problem. Technical report, Carnegie-Mellon University (1976)
7. de Werra, D.: Scheduling in sports. In: Hansen, P. (ed.) *Annals of Discrete Mathematics* (11), North-Holland Mathematics Studies, vol. 59, pp. 381–395. North-Holland (1981)
8. de Werra, D.: Some models of graphs for scheduling sports competitions. *Discrete Applied Mathematics* **21**(1), 47–65 (1988). [https://doi.org/https://doi.org/10.1016/0166-218X\(88\)90033-9](https://doi.org/https://doi.org/10.1016/0166-218X(88)90033-9), <https://www.sciencedirect.com/science/article/pii/0166218X88900339>
9. Di Gaspero, L., Schaerf, A.: A tabu search approach to the traveling tournament problem. In: *Atti della Giornata di Lavoro del gruppo RCRA 2005: Analisi sperimentale e benchmark di algoritmi per l'Intelligenza Artificiale*. pp. 23–27 (2005)
10. Easton, K., Nemhauser, G., Trick, M.: The traveling tournament problem description and benchmarks. In: *Principles and Practice of Constraint Programming CP 2001: 7th International Conference, CP 2001 Paphos, Cyprus, November 26–December 1, 2001 Proceedings* 7. pp. 580–584. Springer (2001)
11. Fujiwara, N., Imahori, S., Matsui, T., Miyashiro, R.: Constructive algorithms for the constant distance traveling tournament problem. In: *Practice and Theory of Automated Timetabling VI*. pp. 135–146. Springer Berlin Heidelberg (2007)
12. Hassel, L., Verduin, K., Løyen, B., Thomson, S.L., Weise, T., van den Berg, D.: The traveling tournament problem: Constraint violations for different maxstreak values. Late-breaking abstract at EvoStar 2025 (2025)
13. Heins, J., Whitley, D., Kerschke, P.: To repair or not to repair? investigating the importance of ab-cycles for the state-of-the-art tsp heuristic eax. In: *Proceedings of the Genetic and Evolutionary Computation Conference*. pp. 231–239 (2025)
14. Horn, R., van den Berg, D., Jansen, R., Verduin, K., van Eck, O.: Randomly sampling hp-protein conformations: Mission impossible? In: *International Joint Conference on Computational Intelligence*. pp. 323–338. Springer (2023)
15. Jansen, R., Horn, R., van Eck, O., Verduin, K., Thomson, S.L., van den Berg, D.: Can hp-protein folding be solved with genetic algorithms? maybe not. In: *IJCCI*. pp. 131–140 (2023)

16. Khelifa, M., Boughaci, D., Aïmeur, E.: An enhanced genetic algorithm with a new crossover operator for the traveling tournament problem. In: 2017 4th International Conference on Control, Decision and Information Technologies (CoDIT). pp. 1072–1077 (2017). <https://doi.org/10.1109/CoDIT.2017.8102741>
17. Kirkman, T.P.: On a problem in combinatorics. *Camb. Dublin Math. J.* **2**, 191–204 (1847)
18. Loyen, B., Bart, D., Richoux, F., van den Berg, D.: The traveling tournament problem: Valid solutions are different across instance sizes. In: Proceedings of the 17th International Joint Conference on Computational Intelligence ECTA (2025)
19. Loyen, B., Hassel, L., Thomson, S.L., Weise, T., van den Berg, D.: The traveling tournament problem: Valid solutions are very different. Late-breaking abstract at EvoStar 2025 (2025)
20. Ribeiro, C., Urrutia, S., de Werra, D.: Combinatorial Models for Scheduling Sports Tournaments. EURO Advanced Tutorials on Operational Research, Springer International Publishing (2023)
21. Rutjanisarakul, T., Jiarasuksakun, T.: A sport tournament scheduling by genetic algorithm with swapping method. *Journal of Engineering and Applied Sciences* **13** (04 2017). <https://doi.org/10.3923/jeasci.2018.5796.5800>
22. Saul, S., Adewumi, A.: An artificial bees colony algorithm for the traveling tournament problem. In: 41st Annual Conference of the Operations Research Society of South Africa. p. 10 (2012)
23. Solberg, H.A., Gaustad, T.: International sport broadcasting: A comparison of european soccer leagues and the major north american team sports. *Sport Broadcasting for Managers* pp. 84–102 (2022)
24. Tajbakhsh, A., Eshghi, K., Shamsi, A.: A hybrid pso-sa algorithm for the traveling tournament problem. In: 2009 International Conference on Computers & Industrial Engineering. pp. 512–518 (2009). <https://doi.org/10.1109/ICCIE.2009.5223865>
25. Thielen, C., Westphal, S.: Complexity of the traveling tournament problem. *Theoretical Computer Science* **412**(4-5), 345–351 (2011)
26. Uthus, D.C., Riddle, P.J., Guesgen, H.W.: An ant colony optimization approach to the traveling tournament problem. In: Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation. p. 8188. GECCO '09, Association for Computing Machinery, New York, NY, USA (2009). <https://doi.org/10.1145/1569901.1569913>, <https://doi.org/10.1145/1569901.1569913>
27. Verduin, K., Thomson, S., van den Berg, D.: Too constrained for genetic algorithms too hard for evolutionary computing the traveling tournament problem. In: Proceedings of the 15th International Joint Conference on Computational Intelligence - ECTA. pp. 246–257 (2023)
28. Verduin, K., Thomson, S.L., van den Berg, D.: Too constrained for genetic algorithms. too hard for evolutionary computing. the traveling tournament problem. In: Proceedings of the 15th International Joint Conference on Computational Intelligence ECTA - Volume 1. pp. 246–257 (2023). <https://doi.org/10.5220/0012192100003595>
29. Verduin, K., Weise, T., van den Berg, D.: Why is the traveling tournament problem not solved with genetic algorithms? Late-breaking abstract at EvoStar 2023 (2023)
30. Wikipedia: Sports in the united states (2025), https://en.wikipedia.org/wiki/Sports_in_the_United_States, (Consulted 29 October 2025)
31. Yamaguchi, D., Imahori, S., Miyashiro, R., Matsui, T.: An improved approximation algorithm for the traveling tournament problem. *Algorithmica* **61**, 1077–1091 (2011)